

Unsupervised learning 2026 – lecture slides (BERN02)

Unsupervised learning – lecture slides (BERN02)

- ISL: 12.1, 12.2 (PCA), 12.4 (clustering)
- Relevant recordings from ISL:
 - 12.1 Principal components analysis <https://youtu.be/kpuQgOzQXfM?si=jdTOzBq77rxkRfQo>
 - 12.2 Higher order principal components <https://youtu.be/O30nHhyBiAs?si=WjYxUOHEudLvHujh>
- StatQuest: <https://www.youtube.com/c/joshstarmar> (PCA, k-means, hierarchical clustering, t-SNE)
- Your friendly AI (have a conversation – then check the answer against the lab)
- Lab: `unsupervised_learning.ipynb`

Introduction to Unsupervised Learning

Definition

- Learning from **unlabeled** data $X = \{x_1, x_2, \dots, x_n\}$, each $x_i \in \mathbb{R}^p$
- Goal: discover hidden structure – groups, directions, densities, rules
- No y : nothing tells the algorithm whether it is right

Comparison with the other learning paradigms

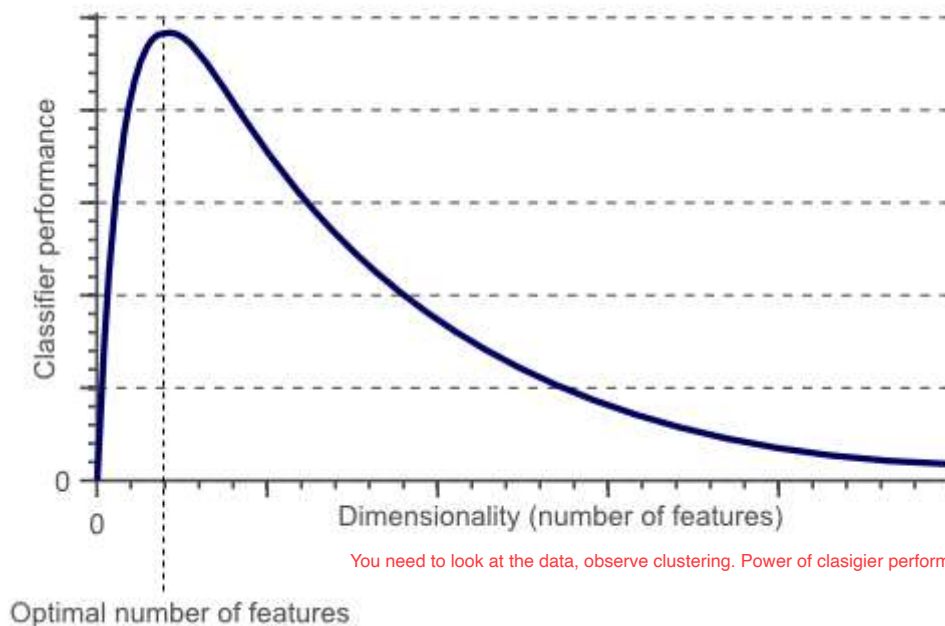
Classic challenge: how many clusters are there?

paradigm	input	what is learned	example in chemistry
Supervised	(x_i, y_i)	$f : X \rightarrow Y$	predict toxicity from descriptors
Unsupervised	x_i only	<u>structure</u> of X	how many species are in my spectra?
Self-supervised	x_i , labels made from x itself	representation	predict masked part of a spectrum
Reinforcement	actions, rewards	policy	optimise a synthesis route

Ex. of data labeling: image processing to make borders around clusters and count them

Key challenges

- **No ground truth** – you cannot compute an error; you judge by usefulness, stability and domain knowledge
- 1 • **How many?** clusters, components – the data do not tell you directly (but see silhouette, scree plot)
- 2 • **Scale** – distances depend on the units of each variable (cm vs kg): scaling is a scientific decision
- 3 • **Curse of dimensionality** – in p dimensions, $d(p, q) = \sqrt{\sum_{i=1}^p (p_i - q_i)^2}$ grows with every added dimension, all points become "far" and equally far



Types of unsupervised learning tasks

1. **Clustering** – group similar observations
2. **Dimension reduction** – find a few directions (linear) or a low-dimensional map (non-linear) that carry the information
3. **Density estimation** – with a given form (Gaussian mixture) or without (kernel density, histogram)
4. **Association rule learning** – which items occur together
5. **Generative models** – learn to produce new data that look like the old (autoencoders, diffusion) – next lectures

Today: 1 and 2 in depth, 3 and 4 briefly.

Clustering

- Definition: partition the observations into clusters $C = \{C_1, C_2, \dots, C_k\}$ so that points within a cluster are similar and points in different clusters are not
 - "Similar" means: a **distance** – Euclidean by default, and that is where scaling enters
 - Families (Linda covered these in depth in BERN01):
 1. Exclusive / centroid based (k-means, k-medoids)
 2. Hierarchical (agglomerative, divisive)
 3. Density based (DBSCAN, OPTICS)
 4. Probabilistic (Gaussian mixture models – "soft" k-means)
 5. Overlapping / fuzzy (fuzzy c-means: memberships that sum to 1)
 6. Graph based (spectral clustering)
-

K-means clustering

- Objective: minimise the within-cluster sum of squares

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$
- Algorithm (Lloyd):
 1. Initialise k centroids (k-means++: spread them out)
 2. Assign every point to the nearest centroid
 3. Move every centroid to the mean of its points
 4. Repeat 2–3 until nothing changes
- Every step lowers J , so it converges – but to a **local** minimum: run it several times (`n_init`) and keep the best
- Complexity $O(n k p i)$: n points, k clusters, p dimensions, i iterations – cheap
- Assumes **convex, similar-sized blobs**: it partitions the plane into Voronoi cells

<https://www.naftaliharris.com/blog/visualizing-k-means-clustering/>

How many clusters? (the main challenge)

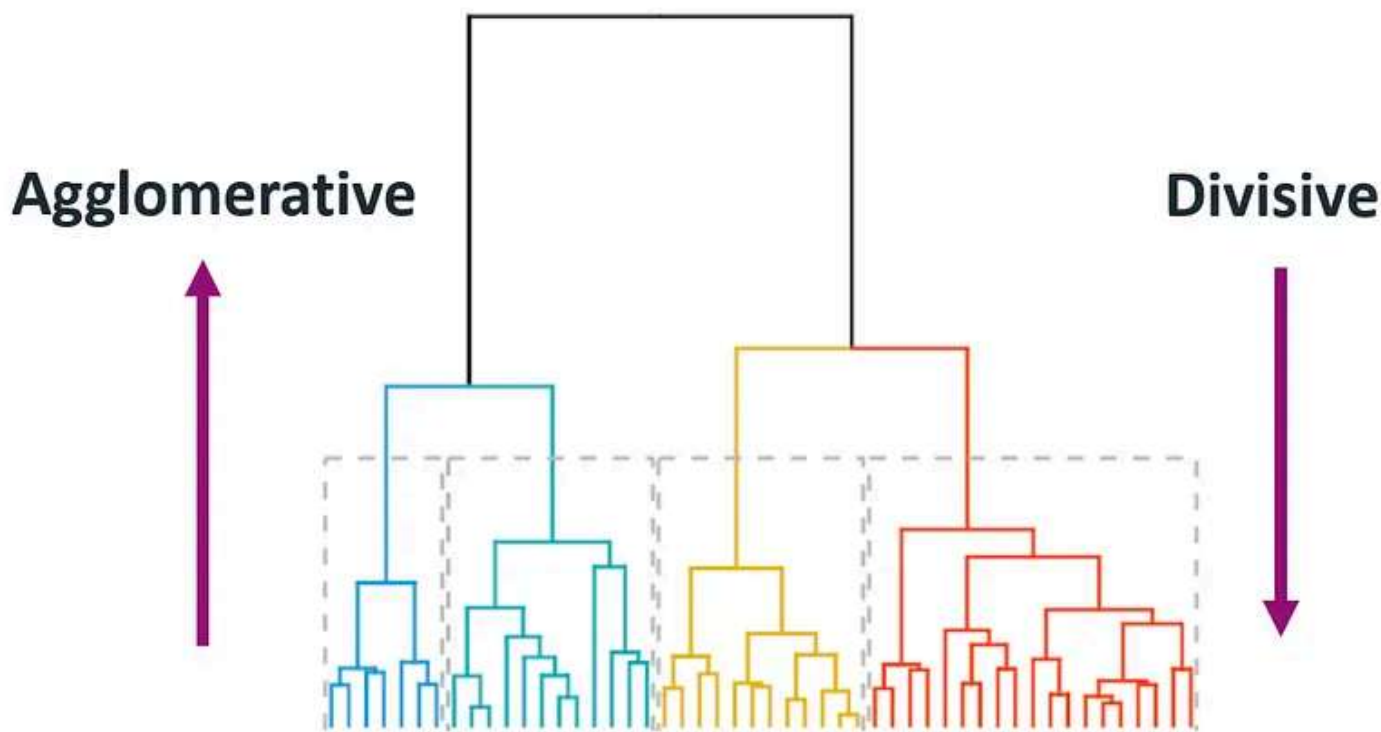
- **Elbow**: plot $J(k)$; J always falls with k , look for the bend
- **Silhouette**: for each point i , a_i = mean distance to its own cluster, b_i = mean distance to the nearest other cluster

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \in [-1, 1]$$
 - $s_i \approx 1$: well inside its cluster; $s_i \approx 0$: on a border; $s_i < 0$: probably in the wrong cluster
 - scan $k = 2 \dots 10$, take the maximum of the mean silhouette (or the smallest k on a plateau)
- **Dendrogram** (hierarchical): a long vertical stretch without merges = well separated clusters

- **Domain knowledge** beats all three: how many species do I expect?
- Lab: iris says $k = 2$ by silhouette, although there are 3 species – the honest answer of the data

Hierarchical clustering

Agglomerative: every point starts as its own cluster; merge the two closest clusters; repeat until one is left. The merge history is the **dendrogram**; cutting it at a height gives a partition – no k needed in advance.



<https://www.linkedin.com/pulse/hierarchical-clustering-comprehensive-guide-understanding-massimo-re-r73gf>

Linkage: what does "closest clusters" mean?

1. Single linkage (nearest neighbour) – follows chains and curved shapes, sensitive to noise bridges

$$d(C_i, C_j) = \min_{x \in C_i, y \in C_j} d(x, y)$$

2. Complete linkage – compact clusters, least sensitive to outliers

$$d(C_i, C_j) = \max_{x \in C_i, y \in C_j} d(x, y)$$

3. Average linkage – in between

$$d(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x \in C_i} \sum_{y \in C_j} d(x, y)$$

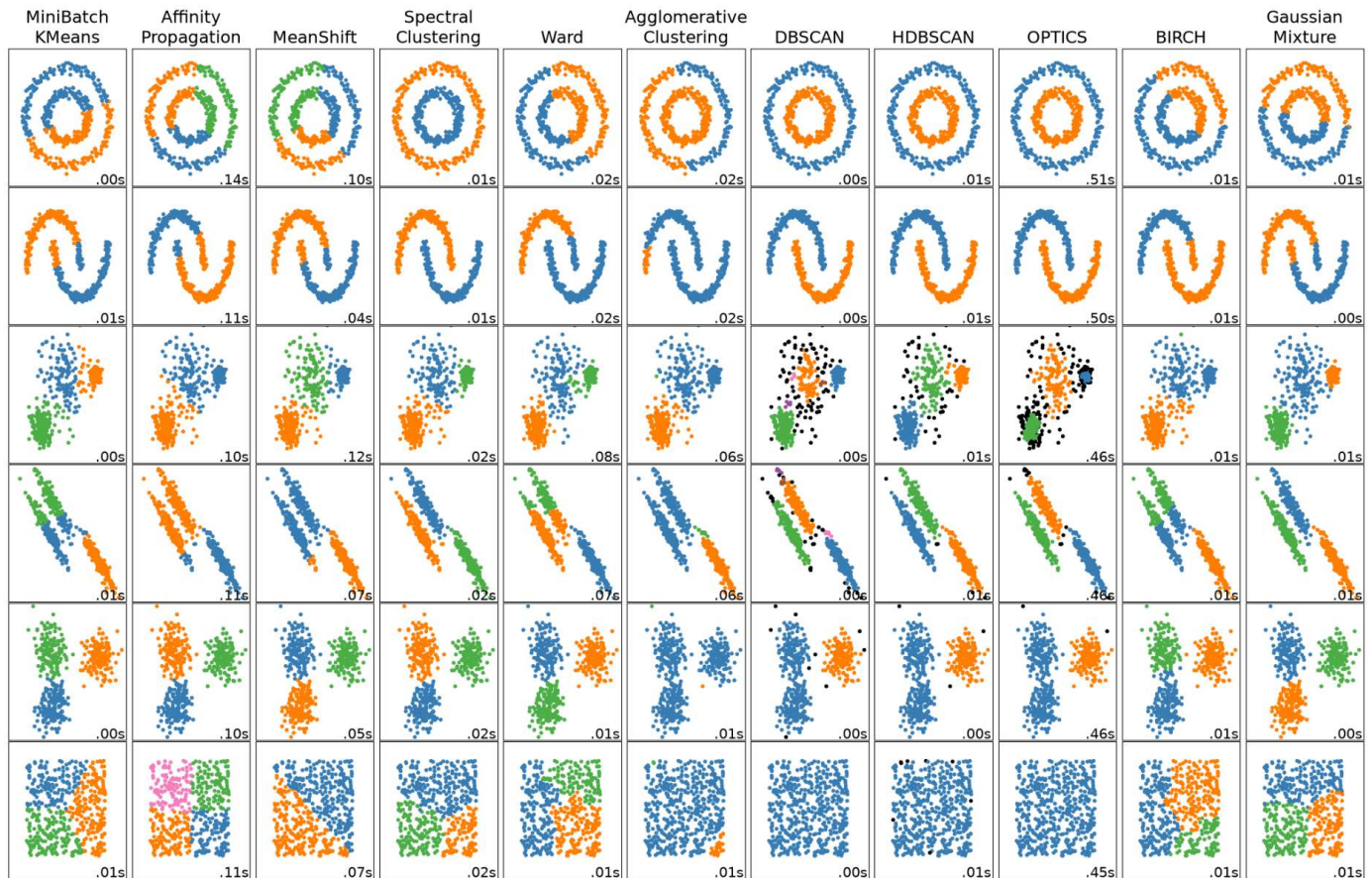
4. Ward's method – merge the pair whose merge increases the total within-cluster variance least; gives k-means-like compact clusters

Same data, four linkages, four different trees – the lab shows this on iris.

Density based: DBSCAN

- Two parameters: radius `eps` and `min_samples`
- A point with $\geq$ `min_samples` neighbours within `eps` is a **core point**; connected core points form a cluster; points reachable from a core point join it; everything else is **noise**
- No k needed; finds clusters of **any shape**; labels outliers explicitly
- Needs a density contrast: fails when clusters have very different densities, or when they touch
- Rule of thumb for `eps`: a few times the typical nearest-neighbour distance

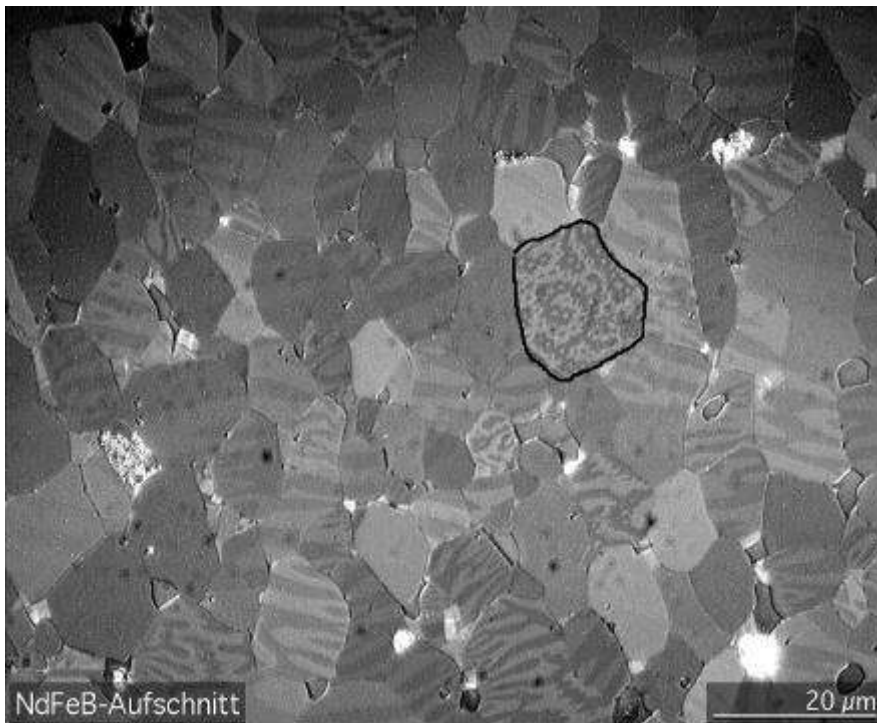
Which algorithm for which data?



https://scikit-learn.org/stable/auto_examples/cluster/plot_cluster_comparison.html

- Convex blobs: k-means, Ward, Gaussian mixtures
- Rings, moons, chains: DBSCAN, single linkage, spectral clustering
- Very different densities: OPTICS, HDBSCAN
- Lab: the smiley point clouds – k-means cuts the ring into pieces, DBSCAN finds it whole

Clustering in the lab: magnetic domains



Gorchy – Own work, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=4459327>

- Observations = pixels; features = intensity (+ position, + local texture)
- Segmentation *is* clustering; pre-processing (binning, smoothing, edge filters) often matters more than the algorithm

Dimensionality Reduction

Purpose: map the feature space from p to M dimensions ($M < p$) while keeping the information

1. **Linear** techniques – new axes are linear combinations of the old (PCA, LDA, factor analysis)
2. **Non-linear** techniques – a map that preserves neighbourhoods, not distances (t-SNE, UMAP, Isomap, LLE)
3. **Neural-network** techniques – autoencoders (a non-linear PCA with a bottleneck) – next lectures

Linear techniques

1. Principal Component Analysis (PCA) – maximise the variance captured by each new axis; unsupervised
2. Linear Discriminant Analysis (LDA) – maximise the distance between known classes, minimise the variance within; **supervised**
3. Factor analysis – like PCA but models measurement noise per variable and allows rotations

Principal Component Analysis (PCA)

The first principal component is the direction $\phi_1 = (\phi_{11}, \dots, \phi_{p1})$ with unit length that maximises the variance of the projected data:

$$\max_{\phi_1} \left\{ \frac{1}{n} \sum_{i=1}^n \left(\sum_{j=1}^p \phi_{j1} x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^p \phi_{j1}^2 = 1$$

- $\phi_1 = \mathbf{loadings}$ (the direction), $z_{i1} = \sum_j \phi_{j1} x_{ij} = \mathbf{scores}$ (the coordinates)
- The second component: same, but orthogonal to the first; and so on – at most $\min(n-1, p)$ components
- Each component explains the largest possible share of the *remaining* variance

PCA is a rotation of the coordinate system

- In 2-D: rotate the axes by α , the new x' is the candidate component, the new y' is what it does not explain

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \alpha & \sin \alpha \\ -\sin \alpha & \cos \alpha \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

- PC1 = the angle with the smallest $\sum y_i'^2$ (perpendicular residuals) = the largest $\sum x_i'^2$; the total $\sum x'^2 + \sum y'^2$ is constant
- Contrast with **regression**: regression minimises the vertical residuals (y given x , asymmetric); PCA minimises the perpendicular ones (symmetric) – lab part 3
- Remember the curse of dimensions: the rotation lives in p dimensions, we only *look* at 2 or 3

Steps – and the scaling decision

1. **Centre** the data (always): $X \leftarrow X - \bar{x}$ – otherwise the first component is the mean, not a variation
2. **Scale** (decide!): $Z = \frac{X - \mu}{\sigma}$
 - variables in different units (cm, kg, pH) → scale, or the unit choice decides the result
 - variables of the same kind (absorbance at many wavelengths) → usually do **not** scale, the variance carries information
3. Covariance matrix $\Sigma = \frac{1}{n-1} Z^T Z$
4. Eigen-decomposition $\Sigma v = \lambda v$; sort by decreasing λ
5. Project: $Y = ZW$ (scores)

Lab: iris raw → PC1 "explains" 97 % (it is the mean); centred → 92 %; standardised → 73 %. Same data, three answers.

How many components? Scree plot and PVE

- Proportion of variance explained by component m (ISL 12.8, 12.9):

$$\text{PVE}_m = \frac{\sum_i z_{im}^2}{\sum_j \sum_i x_{ij}^2} = \frac{\lambda_m}{\sum_k \lambda_k}$$

- **Scree plot:** PVE (or the singular values) against m ; look for the elbow, or for the point where the values reach a flat noise floor
 - Cumulative PVE: how much do I keep with M components?
 - The dropped components are not "wrong" – they are what you have decided to call noise
-

SVD is the way to compute PCA

Any real matrix factorises as $X = U\Sigma V^T$

- U ($n \times n$): **left singular vectors**, orthonormal, patterns over the observations
- Σ : diagonal, **singular values** $s_1 \geq s_2 \geq \dots \geq 0$
- V ($p \times p$): **right singular vectors**, orthonormal, patterns over the features

For centred X : $X^T X = V \Sigma^2 V^T$, so

- loadings = V , scores = $XV = U\Sigma$, variances $\lambda_m = s_m^2/(n-1)$, $\text{PVE}_m = s_m^2 / \sum_k s_k^2$
 - `U, S, Vt = np.linalg.svd(X, full_matrices=False)` – numerically stable, no covariance matrix needed
 - Signs of u_m, v_m are arbitrary (flip both) – interpret sign *patterns*, not signs
-

Truncated SVD = low-rank approximation = denoising

- Keep the first k singular triplets: $X_k = U_{:,1:k} \Sigma_{1:k} V_{:,1:k}^T$
 - X_k is the best rank- k approximation of X in the least-squares sense (Eckart–Young)
 - The residual $X - X_k$ should look like noise; its standard deviation tells you the noise level
 - Related tools: `np.linalg.lstsq` solves least squares via SVD (pseudo-inverse $X^+ = V\Sigma^{-1}U^T$) and handles rank-deficient problems; `np.linalg.matrix_rank` counts the singular values above a tolerance
-

SVD on a spectro-kinetic dataset (lab part 4)

Transient absorption: $D(t, \lambda) = \sum_j c_j(t) \varepsilon_j(\lambda)$, i.e. $D = C E^T$ – exactly the form SVD decomposes

Model: $\text{GS} \xrightarrow{h\nu} \text{A} \xrightarrow{0.1 \text{ ps}} \text{B} \xrightarrow{10 \text{ ps}} \text{C} \xrightarrow{1 \text{ ns}} \text{GS}$, bleach $c_{\text{GS}} = -(c_{\text{A}} + c_{\text{B}} + c_{\text{C}})$

What SVD tells you:

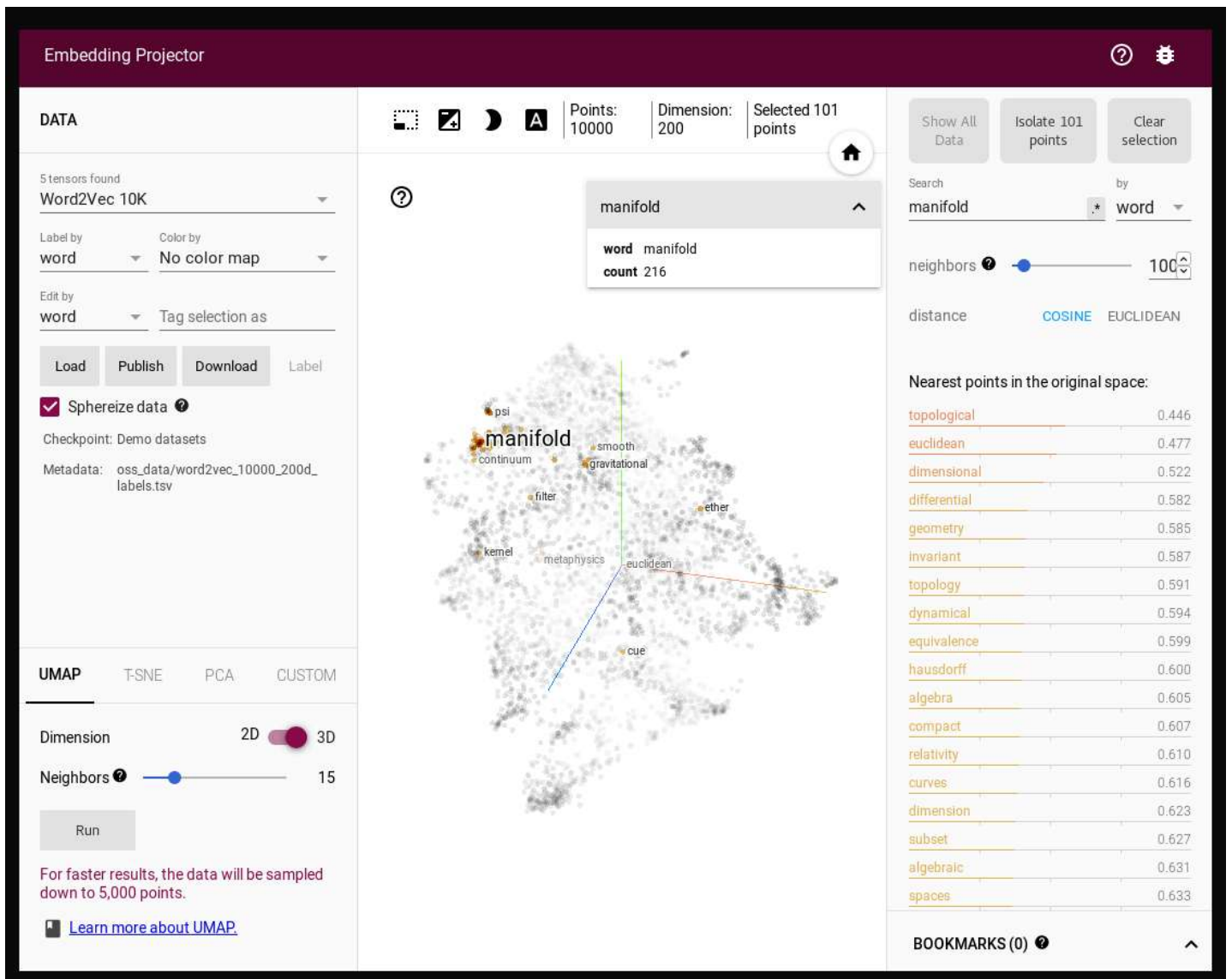
- the **rank** – how many independent components; here **3, not 4**: the mass balance makes the bleach linearly dependent (difference spectroscopy can never show more than $n_{\text{species}} - 1$ from the rank)
 - the **noise level** – singular values that sit on a flat floor
 - a **denoised** data matrix D_3
-

What SVD cannot tell you: rotational ambiguity

- $D = U\Sigma V^T = (U\Sigma T)(T^{-1}V^T)$ for **any** invertible T : infinitely many factorisations fit equally well
 - SVD picks the orthonormal one ordered by variance – real spectra are neither orthogonal nor ordered, so v_m are **mixtures** of species spectra with positive and negative lobes
 - To get chemistry you must add information: a kinetic model, non-negativity, known spectra, ...
 - **Global analysis**: fix a kinetic model $C(t; \tau_1, \tau_2, \tau_3)$, solve the spectra linearly by least squares $E^T = C^+ D$, iterate only the few lifetimes with a non-linear optimiser (variable projection) – this is what KiMoPack / Glotaran do
 - Result: **species-associated difference spectra** $\varepsilon_j - \varepsilon_{\text{GS}}$; the absolute spectra still need the ground-state spectrum measured separately
-

Non-linear techniques

Idea: keep **neighbourhoods**, not distances. Points that are close in p dimensions should be close in 2-D; far apart may be anything.

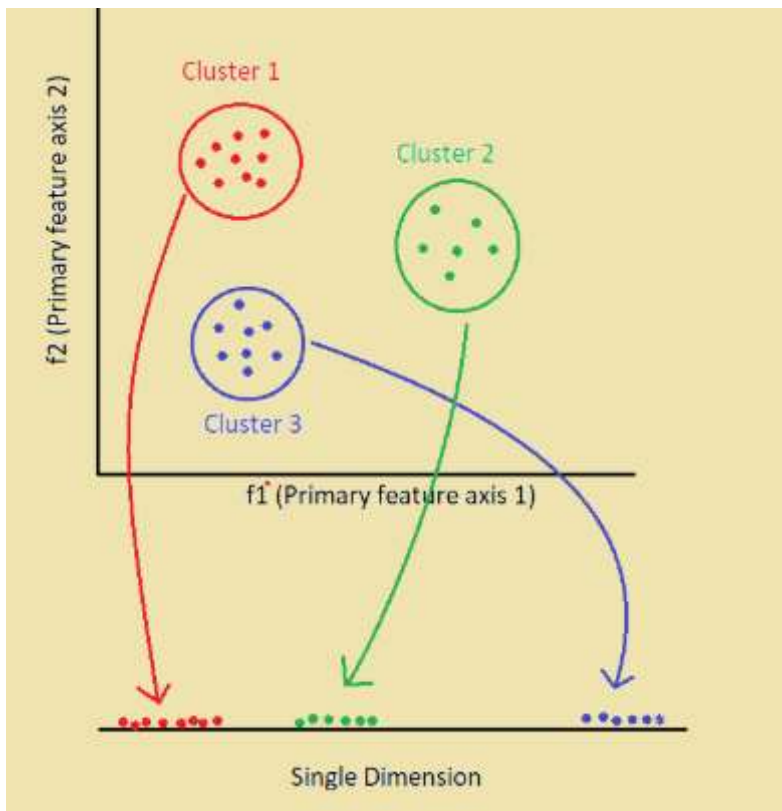


<https://projector.tensorflow.org>

t-Distributed Stochastic Neighbor Embedding (t-SNE)

1. In high dimension: pairwise similarities p_{ij} from Gaussians centred on each point (the `perplexity` sets the effective number of neighbours)
2. In low dimension: similarities q_{ij} from a heavy-tailed Student-t distribution (that is the "t"; it lets distant clusters move far apart)
3. Move the 2-D points by gradient descent to minimise the Kullback–Leibler divergence

$$\sum_{ij} p_{ij} \log(p_{ij}/q_{ij})$$



4.

Read a t-SNE map for **membership only**: cluster sizes, distances between clusters and the axes carry no meaning; a new seed gives a new map.

Uniform Manifold Approximation and Projection (UMAP)

1. Compute the k nearest neighbours of each point
2. Build a weighted neighbourhood graph (a "fuzzy simplicial set")
3. Find the 2-D layout that minimises the cross-entropy between the high- and low-dimensional graphs

Faster than t-SNE, keeps somewhat more global structure, same warnings about interpretation.

```
pip install umap-learn
```

Older manifold methods, for orientation

method	idea	keeps
Isomap	neighbourhood graph → geodesic distances → multidimensional scaling	global geodesic distances
Locally Linear Embedding (LLE)	reconstruct each point from its neighbours, keep the weights in 2-D	local linear structure
Self-organising maps (SOM)	a neural grid that bends itself onto the data	topology

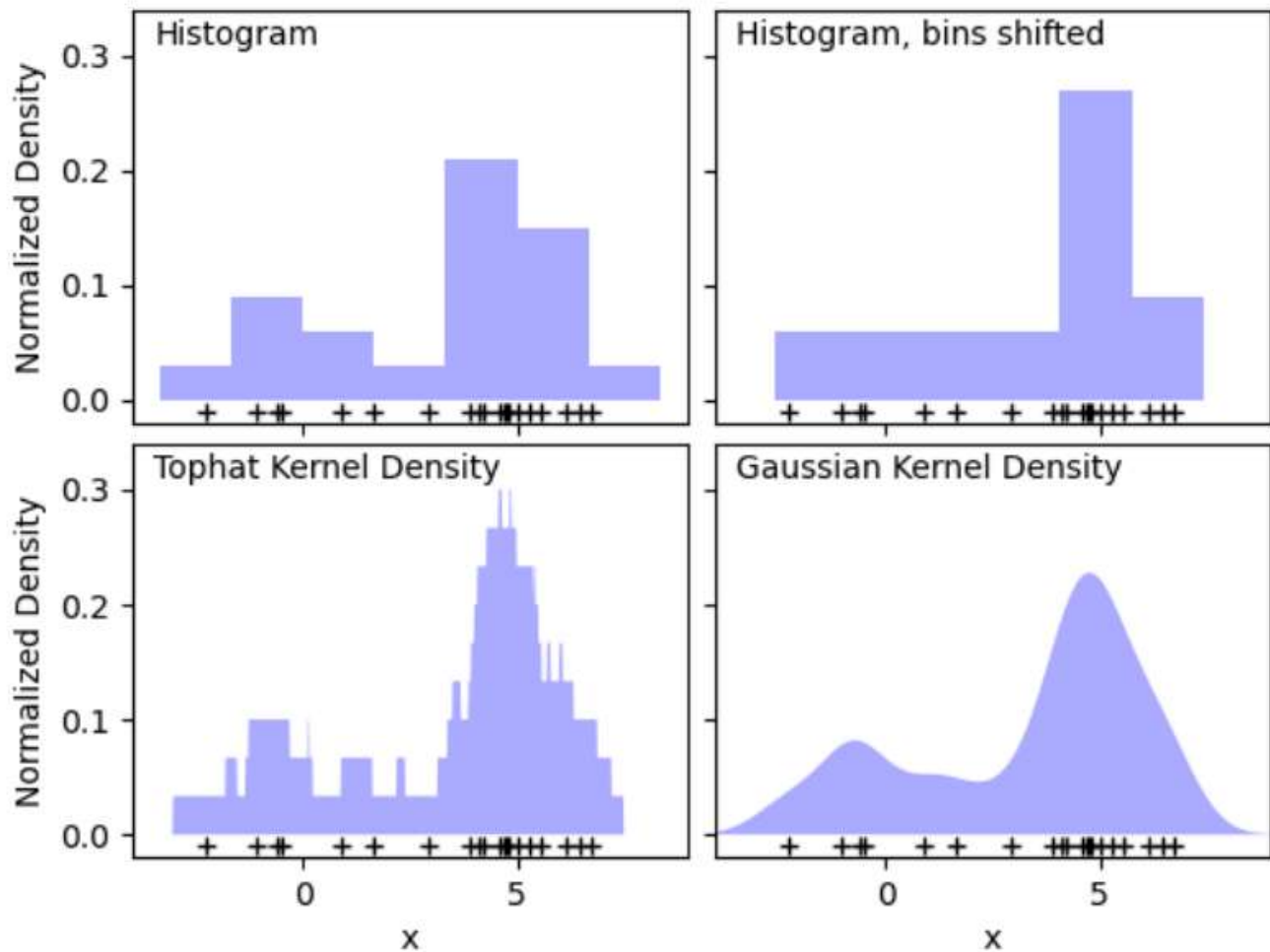
All: choose k neighbours, all $O(n^2)$ or worse, all fail on noisy or disconnected data. Use t-SNE/UMAP for looking, PCA for quantifying.

Neural-network based: autoencoders

- Encoder $p \rightarrow M$, decoder $M \rightarrow p$, trained to reproduce the input: a **non-linear PCA** with a bottleneck of size M
 - With a linear encoder/decoder and squared loss the optimum is PCA
 - Variational autoencoders add a probabilistic bottleneck and become generative
 - <https://www.tensorflow.org/tutorials/generative/autoencoder> – picked up in the neural-network lectures
-

Density Learning

- **With given form (parametric)**: a family of distributions, e.g. Gaussian Mixture Models – the probabilistic cousin of k-means (soft memberships, ellipsoidal clusters, `sklearn.mixture.GaussianMixture`)
- **Without given form (non-parametric)**: histograms, kernel density estimation (a smooth histogram; the bandwidth is the knob)



Straight from <https://scikit-learn.org/stable/modules/density.html>

Association Rule Learning

- Definition: find relationships between variables in large transaction databases ("customers who bought X also bought Y")
- Key concepts for itemsets X, Y in transactions T :
 1. Support – how often the itemset occurs (think: letter frequencies to break a cipher): $supp(X) = \frac{|\{t \in T; X \subseteq t\}|}{|T|}$
 2. Confidence – probability of the consequent given the antecedent (think: word prediction on your phone): $conf(X \Rightarrow Y) = \frac{supp(X \cup Y)}{supp(X)}$
 3. Lift – how much more often than by chance: $lift(X \Rightarrow Y) = \frac{supp(X \cup Y)}{supp(X) \times supp(Y)}$; lift = 1 means independent
- Algorithms:
 1. Apriori – generate candidate itemsets, prune those with low support (every subset of a frequent set is frequent)

2. Eclat – Equivalence Class Clustering and bottom-up Lattice Traversal: count items first, then build sets
 3. FP-growth – compress the transactions into a frequent-pattern tree, then mine the tree
- Chemistry use: co-occurring fragments in mass spectra, co-occurring motifs in reaction databases
-

Summary

- Unsupervised learning finds structure without labels; you judge it by usefulness and stability
 - **PCA/SVD**: centre, decide on scaling, read the scree plot; loadings = V , scores = $U\Sigma$, PVE = $s_m^2 / \sum s_k^2$
 - **SVD on spectra**: gives rank and noise level, denoises – but not species spectra; a model (global analysis) breaks the rotational ambiguity
 - **Clustering**: k-means for blobs, hierarchical for a full picture, DBSCAN for shapes and outliers; choose k with elbow, silhouette or the dendrogram
 - **Non-linear maps** (t-SNE, UMAP): for looking, not for measuring
-

Learning questions

1. What distinguishes unsupervised learning from supervised and reinforcement learning? Give one chemistry example of each.
2. Why is a large number of dimensions a challenge for distance-based methods?
3. Clustering:
 1. Describe the main idea of clustering in a few words and name at least three algorithms from different families.
 2. What is the common main challenge in clustering, and name two quantitative ways to address it.
 3. Why does k-means fail on a ring around a blob, and which method would you use instead?
 4. Why does scaling the variables change a k-means result?
4. Dimensionality reduction:
 1. What is the key idea of PCA? What are loadings and scores?
 2. Why must the data be centred before PCA, and when should they also be scaled?
 3. How is SVD related to PCA (loadings, scores, variances)?
 4. Explain the relation between PCA and linear regression in two dimensions.
 5. A transient-absorption matrix of four species has rank three. Why?
 6. Why are singular vectors in most cases not the physically meaningful spectra? How does global analysis resolve this?

7. Name one linear and one non-linear dimension-reduction method and explain what each preserves.
8. Which features of a t-SNE map may be interpreted and which may not?